

Angular HandsOn 5: Reactive Forms and Validation

HANDS-ON 5 [Intermediate]

Reactive Forms ? FormBuilder, FormGroup, FormArray & Custom Validators

Topics Covered:

ReactiveFormsModule Setup[Check] FormBuilder & FormGroup[Check]
FormControl & FormArray[Check] Built-in Validators[Check]
Custom Synchronous Validators[Check] Custom Async Validators[Check]
Dynamic Form Controls[Check]

Reactive forms define form structure in the component class rather than the template ? making them more powerful, testable, and suited to complex scenarios. You will rebuild the enrollment form as a reactive form and add custom validators and dynamic controls.

IMP

ORT

Add ReactiveFormsModule to the component's imports array (standalone) or the module's imports to enable FormBuilder, FormGroup, and FormControl.

Task 1: Build a Reactive Form with FormBuilder

Goal: Define and bind a reactive form using FormBuilder.

Steps:

48. Generate a ReactiveEnrollmentFormComponent: `ng generate component pages/reactive-enrollment-form`. Add a route at `/enroll-reactive`.

49. Inject FormBuilder in the constructor: `constructor(private fb: FormBuilder) {}`. In `ngOnInit`, build the form: `this.enrollForm = this.fb.group({ studentName: ['', [Validators.required, Validators.minLength(3)]], studentEmail: ['', [Validators.required, Validators.email]], courseId: [null, Validators.required], preferredSemester: ['Odd', Validators.required], agreeToTerms: [false, Validators.requiredTrue] });`

50. Bind the form in the template using `[formGroup]='enrollForm'`. Bind each input using `formControlName='studentName'` (etc.). Note: `ngModel` is needed in reactive forms.

51. Add a Submit button: `<button [disabled]='enrollForm.invalid'>Submit</button>`. On submit, log `enrollForm.value` and `enrollForm.getRawValue()`.

52. Explain in a comment the difference between `enrollForm.value` (excludes disabled controls) and `enrollForm.getRawValue()` (includes all controls).

Hint:

- In reactive forms, the form model lives entirely in TypeScript ? the template just binds to it. This makes the form logic fully unit-testable without a DOM.

- `Validators.requiredTrue` specifically validates that a checkbox is checked ? `Validators.required` only checks for a non-empty/non-null value.

Expected Outcome: Form renders correctly bound to the reactive model. Submit logs the form value. Disabled

Angular HandsOn 5: Reactive Forms and Validation

Submit button enables only when all validators pass.

Task 2: Custom Validators and FormArray for Dynamic Controls

Goal: Write a custom synchronous validator and use FormArray for repeating controls.

Steps:

53. Write a custom validator function `noCourseCode(control: AbstractControl): ValidationErrors | null` that returns `{ noCourseCode: true }` if the control value starts with 'XX' (a disallowed prefix). Apply it to the `courseId` control alongside `Validators.required`.

54. Display the custom error: `<span *ngIf='enrollForm.get("courseId")?.errors?.["noCourseCode"]'>Course code starting with XX is not allowed.</span>`.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book

55. Write an async validator `simulateEmailCheck` that returns a Promise: after 800ms, if the email contains 'test@', return `{ emailTaken: true }`, otherwise return null. Apply it as the third argument of the email `FormControl`: `this.fb.control("", [Validators.required, Validators.email], [simulateEmailCheck])`.

56. Add a `FormArray` for `additionalCourses`: `this.fb.array([])`. Add an 'Add Another Course' button that pushes a new `FormControl` to the array. Render the array: `<div *ngFor='let ctrl of additionalCourses.controls; let i=index'><input [formControl]='ctrl'><button (click)='removeCourse(i)'>Remove</button></div>`.

57. Implement `get additionalCourses()` `{ return this.enrollForm.get('additionalCourses') as FormArray; }` as a typed getter ? explain why this getter is better than casting in the template.

Hint:

- Async validators fire after all sync validators pass (to avoid unnecessary API calls). They return `Observable<ValidationErrors | null>` or `Promise<ValidationErrors | null>`.

- `FormArray` is Angular's answer to dynamic, repeating form sections ? like adding multiple phone numbers or, in this case, enrolling in multiple courses at once.

Expected Outcome: Entering 'XX101' in `courseId` shows the custom error. `test@example.com` shows 'Email taken' after 800ms. Additional course controls can be added and removed dynamically.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book